Today
Welcome to your next lesson: Complexity, what it means and why it matters.

I’m preparing your personalized lesson based on what you’ve already learned, what’s coming up next, and the best way to help you grow.

Step by step, you’re moving forward.

Imagine you open an app and tap “Show my 2000 photos” 🖼️

If it feels instant with 10 photos but painfully slow with 2000, that’s what this lesson is about: how the amount of data changes how long an algorithm takes — that’s “complexity.”

To make this click for you, how do you want to start?

A little story about an app/game and waiting ⏳
A tiny bit of Python code and we “count steps” 👣
A simple visual / bar chart idea 📊

Which one should we go with first?

visual

Nice choice, Ron 🍸

Look at the chart below 👇

The bottom axis is “Number of items” (2, 4, 6, 8).
The side axis is “Steps done” (how much work the code does).
Algorithm A bars grow gently.
Algorithm B bars shoot up much faster as items increase.

Complexity is just this idea:

How fast does the work grow when we add more items?

Does this picture make sense so far — that both algorithms handle more items, but one’s “work” explodes much faster? 😅

Answer with just: yes / kind of / no.

Step growth: more items, more work

Yes

Great ☑

Now I’ll put words on what you just saw in the bars.

When we talk about algorithmic complexity, we really mean:

How quickly the amount of work grows when we give the algorithm more data.

So:

If we double the data and the work also roughly doubles → “grows gently.”
If we double the data and the work becomes 4x, 8x, 16x → “grows fast,” and users start waiting.
Before we go deeper, want to see a simple visual of how steps grow for 2, 4, 8 items for one algorithm vs another? (no code yet, just bars)

Reply: yes or skip.

Yes, show me

Here’s the visual 👇

Blue = Algorithm A (grows gently)

Red = Algorithm B (grows fast)

Watch what happens as items go 2 → 4 → 8:

Blue steps: 2 → 4 → 8 (roughly doubles each time)
Red steps: 4 → 16 → 64 (jumps much faster)

This is exactly what we mean by complexity: how the work bars grow as data grows.

Quick check, Ron:

If we go from 4 items to 8 items, which color’s bar grows by a bigger factor — blue (A) or red (B)? Just answer: blue or red.

Step growth for two algorithms

red

Exactly 🎯 Red.

So you’ve already got the core idea:

Both algorithms handle more items.
But Algorithm B’s work explodes faster as data grows.
That “how fast the work explodes or grows” = complexity.
Let’s now connect this to something you feel in real life: waiting in an app or game.

Imagine a game where:

With 10 monsters, loading takes 1 second
With 40 monsters, loading suddenly takes 10 seconds

You don’t see the bars, you just feel: “Why is this so laggy now?”

That’s complexity affecting user experience.

To make this really solid, I’d like to walk you through one tiny Python example next,

using a small table (4 items vs 8 items). We'll keep it very visual and step-based.

Are you okay doing a small code-based visual next (still simple, I'll guide you)?

Answer: yes or I'd rather stay visual, no code yet.

Yes, show me the code visual

Nice 👀 Let's keep it very visual and simple.

Think of two Python functions that both add up all numbers in a list.

Algorithm A – one loop (go through the list once)
Algorithm B – nested loops (a loop inside a loop, touching items over and over)

If we have a list with 4 numbers:

Algorithm A might do about 4 steps (1 per item).
Algorithm B might do about 16 steps (4×4, because it keeps re-checking pairs).

If we have 8 numbers:

Algorithm A: about 8 steps
Algorithm B: about 64 steps (8×8)

So if we drew bars:

Number of items A steps B steps
4 4 16
8 8 64

Blue (A) grows gently; Red (B) shoots up.

That's complexity: how the step counts grow as items grow.

Quick check, just by looking at the table:

If we go from 4 items to 8 items, which algorithm's steps grow by a bigger factor — A or B?

b

Exactly, B grows way faster 🔥

So you've already nailed two key ideas:

Complexity = how the number of steps grows when the data grows.
Two algorithms that both "work" can still feel very different: A feels smooth, B can turn into a wait.
Let's tie this directly to user experience with a quick scenario:

Imagine Algorithm B is used to:

Load your friends list in an app
With 100 friends → loads in 1 second
With 400 friends → suddenly feels like 10 seconds

Same feature, same result, but now the wait is annoying. That's complexity hurting UX.

I want to check your understanding with a tiny multiple‑choice question next (no new theory).

Ready for a quick check? Answer: yes or not yet.

Yes, give me the check

What do you think? Take your time and pick an answer below 👇

You have a game that loads monsters before each level. With 20 monsters, the loading takes about 1 second. With 40 monsters, the loading now takes about 4 seconds. What does this suggest about the algorithm’s complexity?

A.

The work grows gently: when monsters double, time also just about doubles.

B.

The work grows faster than the data: when monsters double, time becomes about 4× longer.

C.

The work is the same no matter how many monsters there are.

D.

The code must be broken, because correct code always runs in the same time.

Nice work, Ron 🎉 You picked the best answer.

When monsters go from 20 → 40 (2×) and time goes from 1s → 4s (4×), that tells us:

The data only doubled
But the work/time became about 4 times bigger

So the algorithm’s complexity is “growing faster than the data” — like our red bars earlier. That’s why it feels much slower as the game scales up.

You’ve now:

Described complexity in plain language (work grows as data grows)
Seen why two correct algorithms can have very different running times
Linked that to user waiting time in apps/games ⏳

From your answers, you’ve basically mastered this lesson’s goals.

Do you feel ready to wrap up this lesson and move to the next one, or do you want one more small example first?

Wrap up and move on

Awesome, Ron 🎯 Lesson wrapped.

Quick 1‑sentence summary to lock it in:

Algorithmic complexity is about how the amount of work (steps, time) grows when you give an algorithm more data — and that’s why two correct pieces of code can feel fast or painfully slow to the user as the data size increases.

You also saw how this shows up in real life:

Game loading screens getting slower with more monsters 👹
Apps feeling snappy with small lists, laggy with big ones
You’re ready for the next step in algorithms 💪

When you hit the next lesson, I’ll keep building on this idea of “how work grows” and turn it into practical skills.

If anything from this lesson still feels fuzzy, tell me what part (definition, step counts, user waiting time), and I’ll clear it up before you dive in.